RMIT Classification: Trusted

COSC2626/2640
Virtualization and Hypervisors

Dr. Qiang Fu
School of computing technologies

RMIT UNIVERSITY

House keeping

- Make sure that you have access to AWS Learner Lab

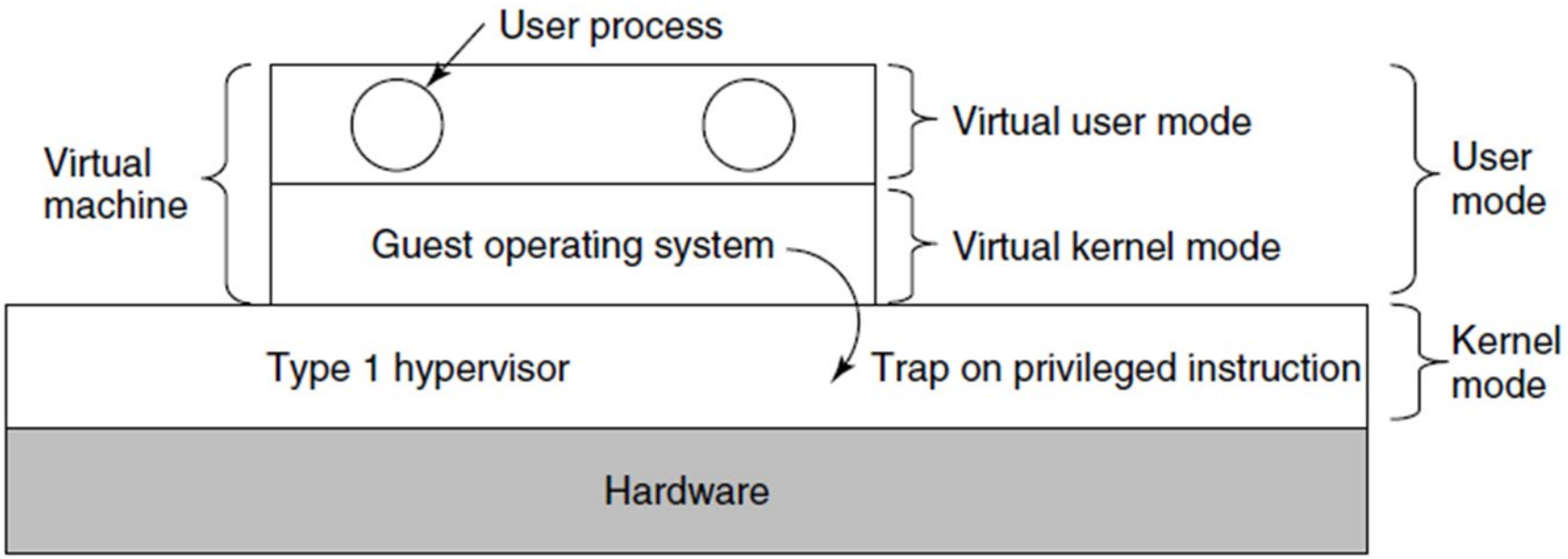
Recap

- Virtual Machine: abstraction of hardware into execution environment
- Hypervisor: type 1 and 2
  - More interested in Type 1 used in the cloud
- Process, user mode vs kernel mode, OS structure
- Virtualization implementation
  - Trap-and-emulate (recap later)

This Lecture

- Virtualization implementation
  - Trap and emulate
  - Memory management
  - Type 1 and 2 hypervisor
- From full virtualization to paravirtualization
- VM migration and virtual appliances
- OS-level virtualization and containers

Virtualization: Trap-and-Emulate



When the operating system in a VM executes a kernel only instruction, it traps to the hypervisor.

Virtualization: memory management

- Memory management: a critical thing to do in a computer system
- Who is going to do this?

Virtualization: memory management

- Memory is divided into pages
- Can your apps/programs see the physical memory?
  - No, they use virtual memory or virtual pages (virtual addresses)
  - Your apps cannot manage memory directly (physical addresses)

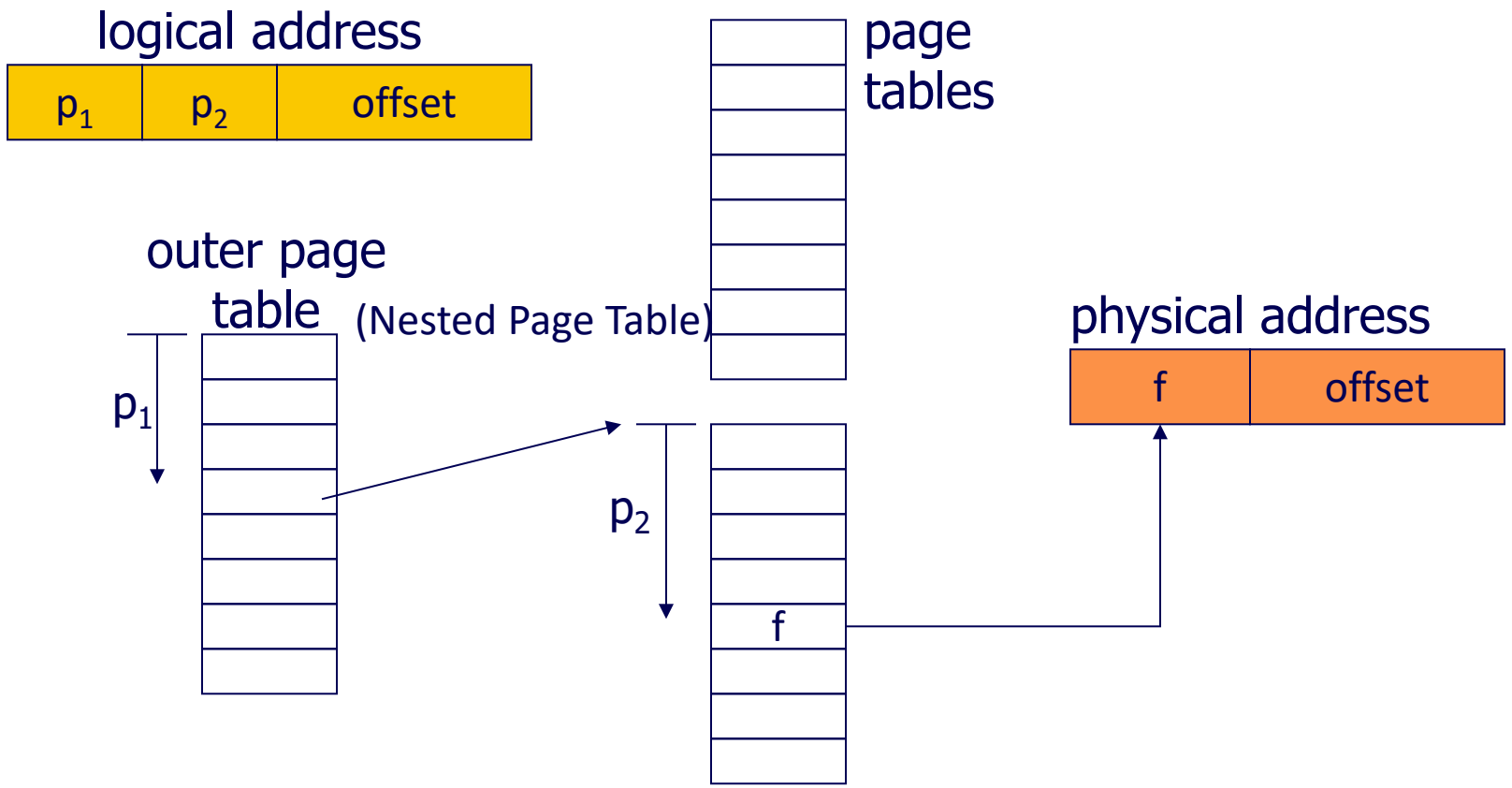
Virtualization: memory management

- OS has to maintain a page table that maps from virtual pages to actual page frames (virtual address → physical address)

- How big is the page table?
  - Very big, think about a 32-bit machine with a page size of 4KB
  - May use multi-level page tables or nested page tables to handle the issue

Virtualization: memory management

A two-level nested page table (NPT)



A page table of page tables

Virtualization: memory management in action

- Virtualization makes memory management even harder and confusing.
  - VM OS still needs to maintain page tables for its VM, but it no longer has direct access to memory
  - Hypervisor also needs to maintain page tables for all VMs.

- You can imagine hypervisor has to work hard and get the page table and mapping right.
  - Provide the actual mapping
  - Avoid conflicts among VMs

Virtualization: Nested Page Tables (NPTs)

- We now have guest virtual address, guest physical address and host physical address
- Eventually, we need to translate from guest virtual address to host physical address

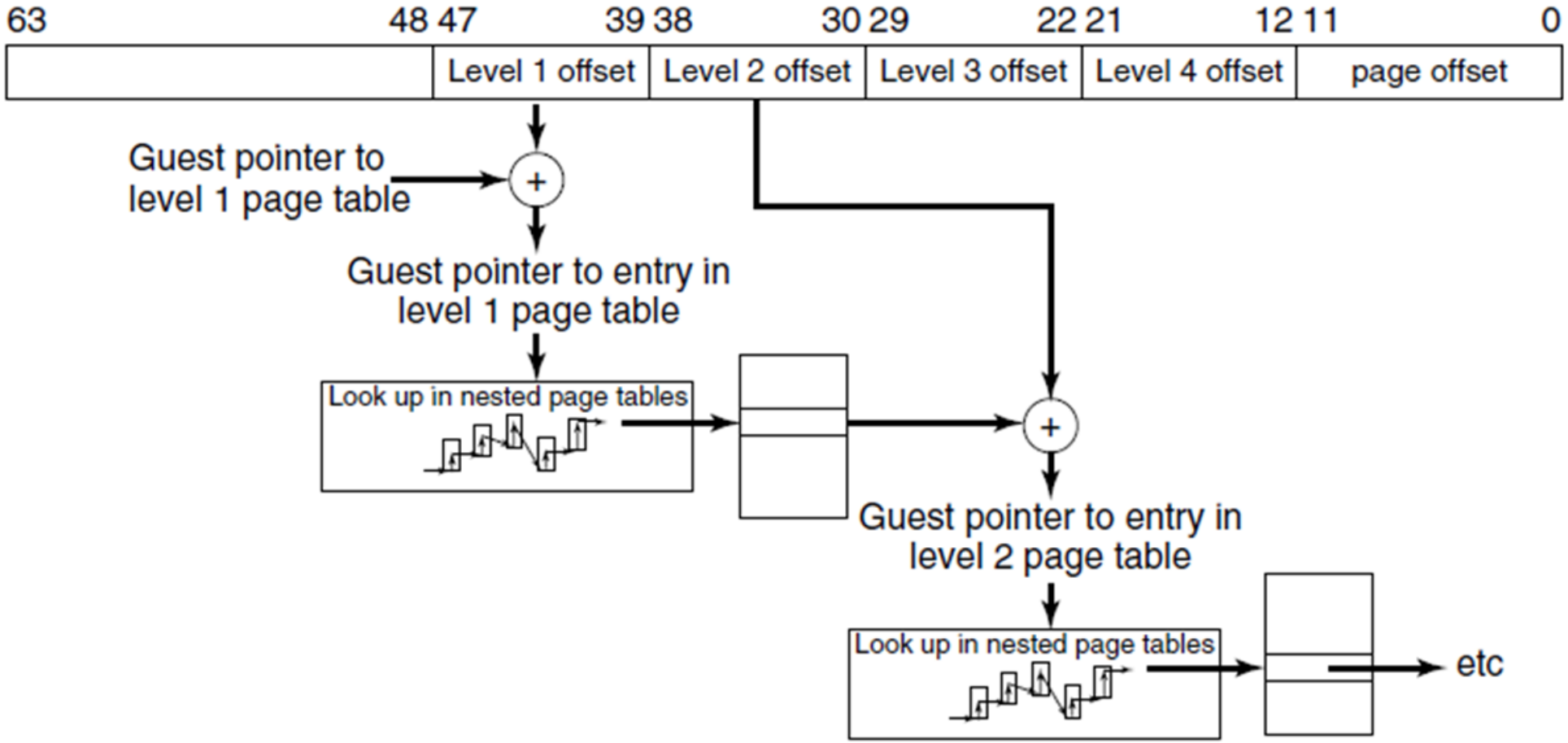
- Each VM maintains page tables to translate guest virtual address to guest physical addresses, business as usual
- Hypervisor maintains per VM NPTs to provide mapping to host physical address
- VM tries to change its page table → Hypervisor makes equivalent change to NPTs

Virtualization: NPTs in hardware

- Every time for a memory access, we need to do:
  - Go through the VM page table to translate the guest virtual address to a guest physical address
  - Also go through the NPTs to finally find the host physical address
- Fortunately, CPUs implement NPTs in hardware
  - hypervisor fully controls the paging
  - CPU accelerates the translation.

Virtualization: NPTs in hardware

- From guest to host physical address
  - Go through the page tables at each level and finally get the page



Virtualization: Type 1 hypervisor

- Commonly found in company datacentres
- In a sense becoming "datacentre operating systems"
  - Run in kernel mode
  - Datacentre managers control and manage OSes in new, sophisticated ways by controlling the Type 1 hypervisor
  - Consolidation of multiple OSes and apps onto less HW
  - Move guests between systems to balance performance
  - Snapshots and cloning

Virtualization: Type 1 hypervisor (category 1)

- Special purpose operating systems that run natively on HW
  - Minimalistic design dedicated for virtualization
  - Rather than providing system call interface, create, run and manage guest Oses (will get back to this)
  - Guests generally don't know they are running in a VM
  - Implement device drivers for host HW because no other component can
  - CPU and memory management

Virtualization: Type 1 hypervisor (category 2)

- General purpose OSes that also provide hypervisor functionality
  - RedHat Enterprise Linux with KVM, Windows with Hyper-V, Oracle Solaris
  - Perform normal duties as well as hypervisor duties
  - Typically, less feature rich than dedicated Type 1 hypervisors
  - In many ways, treat guests OSes as just another process

Virtualization: Type 1 hypervisor (category 1 vs 2)

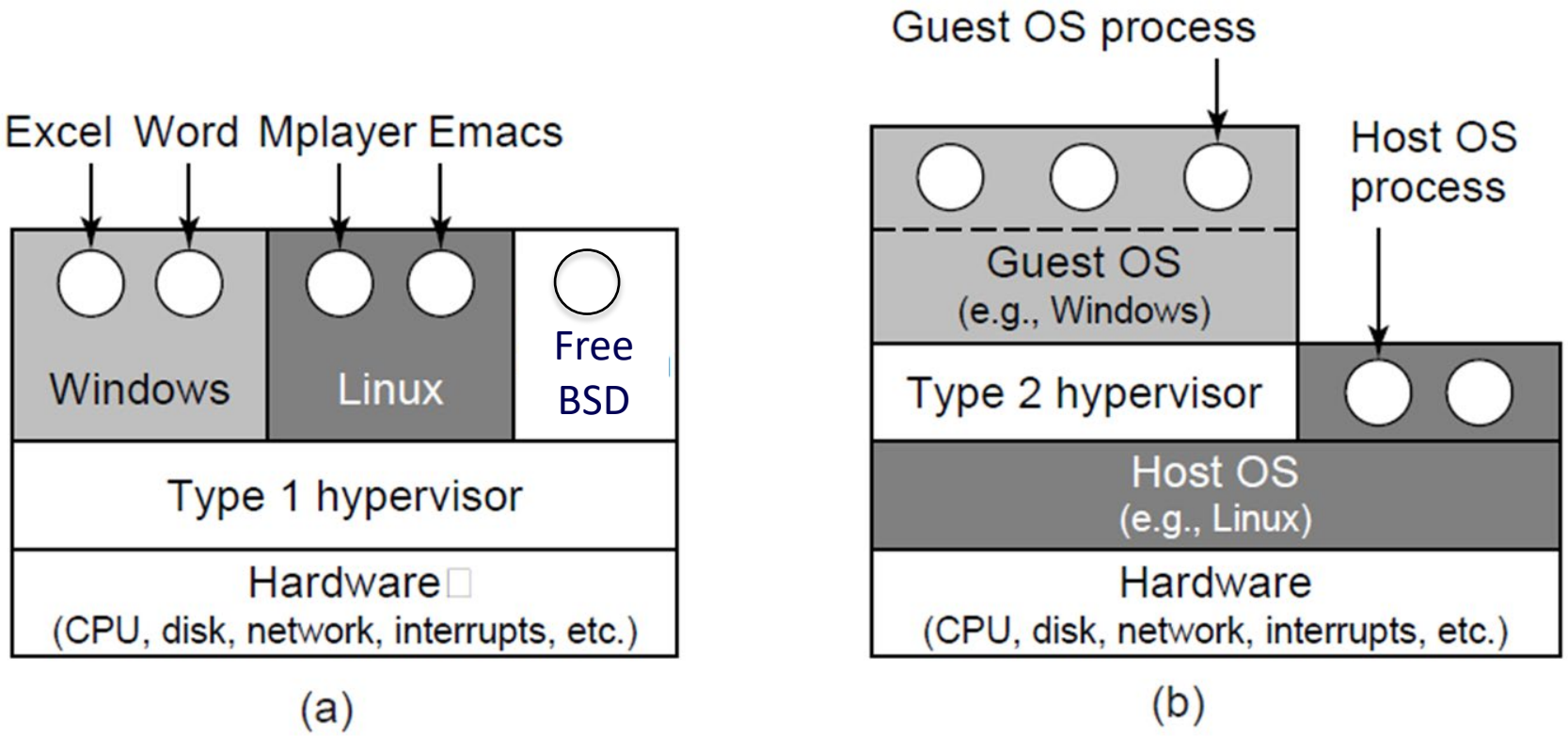
- What do you think?

Virtualization: Type 2 hypervisor

- Run on top of host operating systems
- Less interesting from an OS perspective
  - Very little OS involvement in virtualization
  - Hypervisor is simply another process, run and managed by host
- Even the host doesn't know it is a hypervisor running guests
- Tend to have poorer overall performance because can't take advantage of some HW features
- But also a benefit because require no changes to host OS

Virtualization: Hypervisors

Type 1 and type 2 hypervisors.



Where does Type 1 (category 2) fit?

Paravirtualization: hypervisor vs microkernel

- Recall microkernel is a small kernel that only contains essential OS functionalities
- Type 1 (cat1) hypervisor does the similar thing

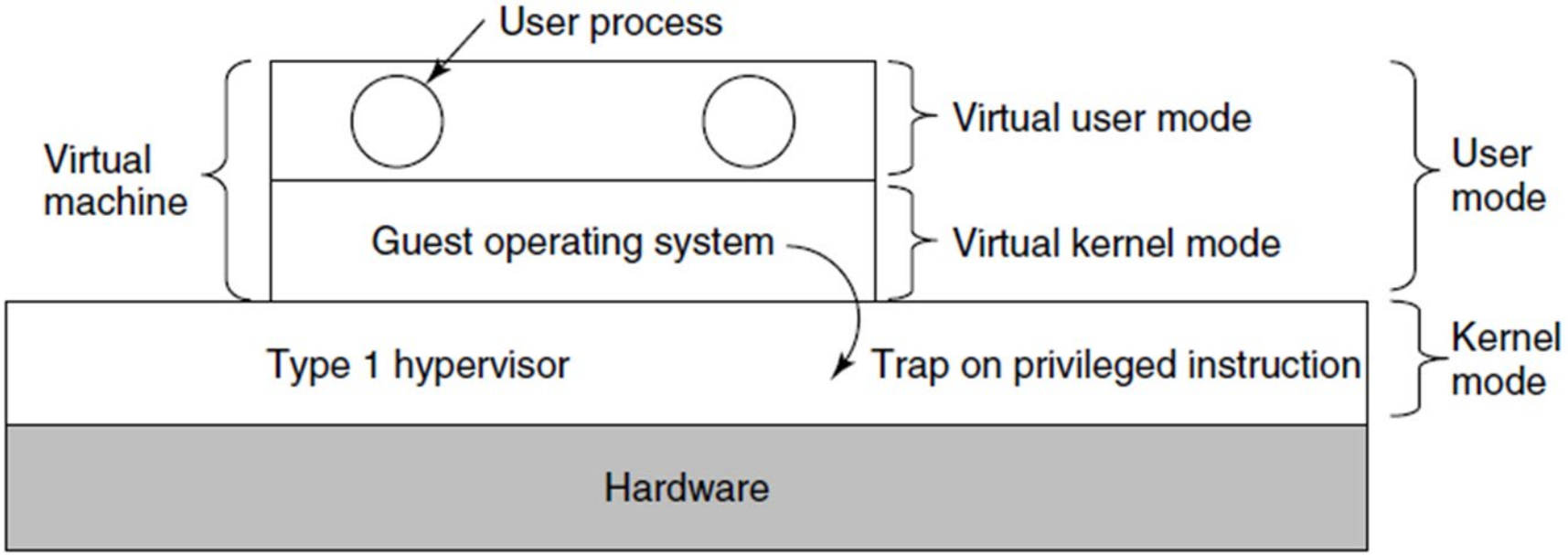
- Hypervisor: trap on privileged instructions
- Microkernel: system call on privileged instructions

Alternative design: Paravirtualization!

Paravirtualization

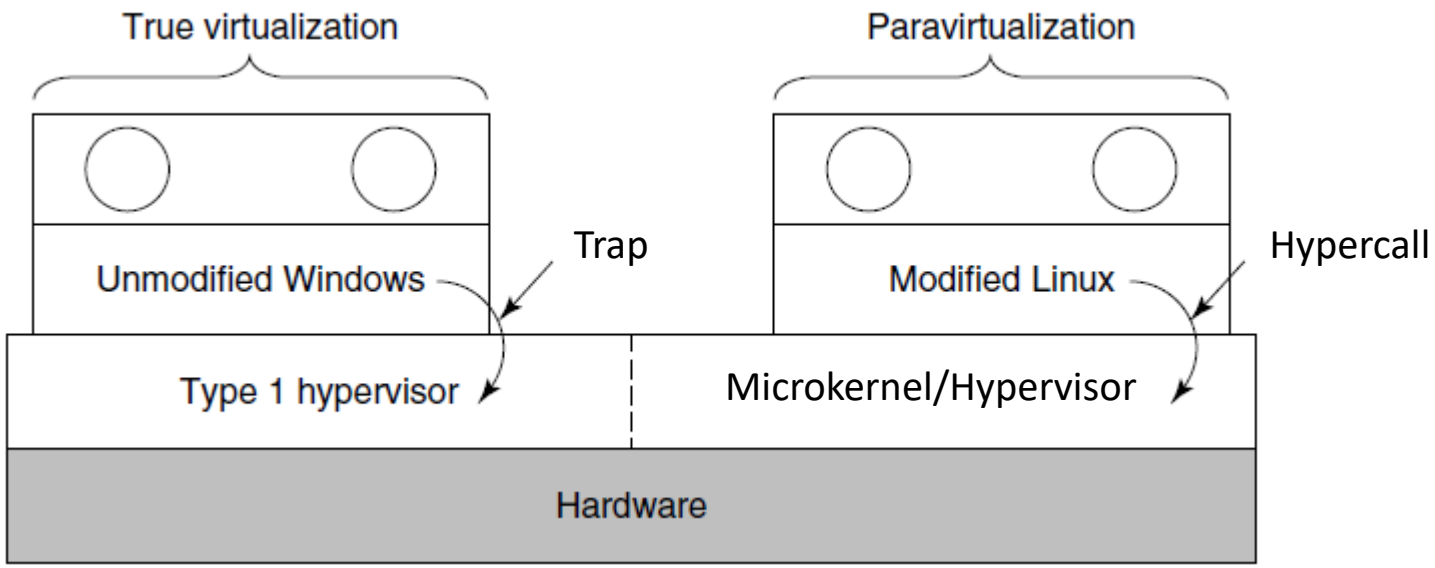
- The guest/VM does not know it is a VM
- It thinks it owns the hardware: trapped on privileged instructions

- Why not let VM know it is just a VM: call on privileged instructions



Paravirtualization

- Guest OS is modified to be hypervisor aware
- Issue a hypercall if need something from hypervisor



True virtualization vs paravirtualization

Paravirtualization

- Paravirtualization allows virtualization without trap-and-emulate
- Increased performance
- Less needed as hardware support for VMs grows

VM Migration

- Hypervisor decouples VMs from physical hardware
- Make it possible to move VMs between systems for whatever reasons

- How to do it?
  - Shut down VMs on the old machine and restart them on the new one?
  - Pause VMs, copy over to the new one, configure and then resume execution?

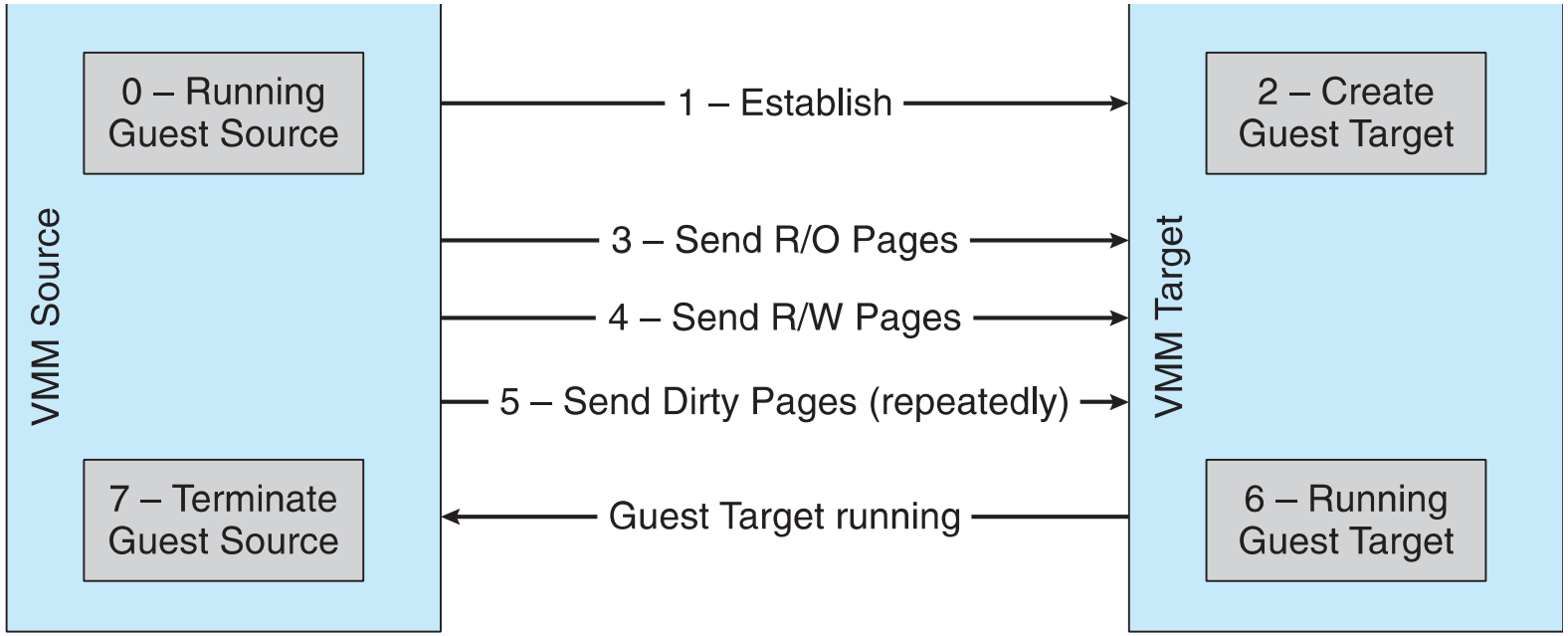
VM Migration

Live migration: move the virtual machine while it is still operational.

- Copy memory pages while the VM is still serving requests.
- A page may be modified after it has already been copied, mark them as dirty.
- When most memory pages have been copied, pause very briefly to copy the remaining dirty pages and resume the VM on the new machine.

VM Migration

Live migration: some details on steps



VM Migration

Live migration: some details on steps

1. Establish connection
2. Create a new VM at target
3. Copy read-only pages
4. Copy read-write pages
5. Copy dirty pages
6. New VM running
7. Terminate the old VM

Are there any other issues?

Checkpointing

- If the state of a VM (e.g., CPU state, memory pages, and storage state) is stored on disk, we have a snapshot of a running machine.
- VM running into a mess, roll back to the snapshot and continue as if nothing happened.

- How to create a snapshot: copy everything?
- Copy-on-write: data is copied only when written.

Virtual Appliances

- Installing new application programs may not be easy
  - Many applications are dependent on numerous other applications and libraries, which are themselves dependent on a host of other software packages, and so on.
  - There may be dependencies on particular versions of the compilers, scripting languages, and the operating system.
- Virtual appliances: create a VM image for installing
  - A software developer can carefully construct a virtual machine, load it with the required operating system, compilers, libraries, and application code, and freeze the entire unit, ready to run.
- Amazon's EC2 cloud has many pre-packaged virtual appliances available for its clients, which it offers as convenient software services ("Software as a Service").

Virtual Appliances

- Do we always need this kind of virtual appliances???

OS-level virtualization

- VMs: hardware virtualization
- VMs are heavyweight, not always needed

- In some instances, what we want to do is:
  - segregate applications,
  - manage their performance and resource use, and
  - create an easy way to start, stop, move, and manage them.

- Can do those things without hardware virtualization
- If the applications are all compiled for the same operating system, OS-level virtualization or application containment is good enough

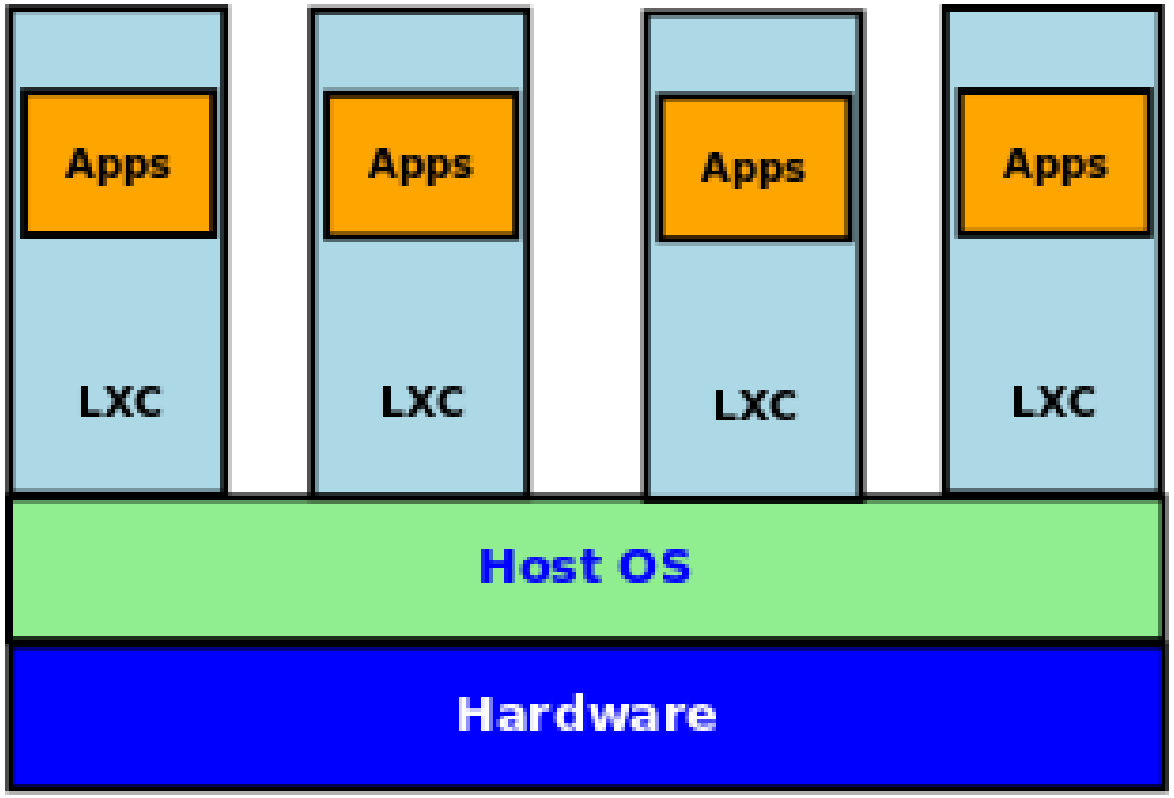
OS-level virtualization

- This gives us the concept of containers
- Only one kernel running: host OS
- Host OS is virtualized and shared among the containers

- Examples: Oracle Solaris containers/zones, Linux Containers (LXC), Docker

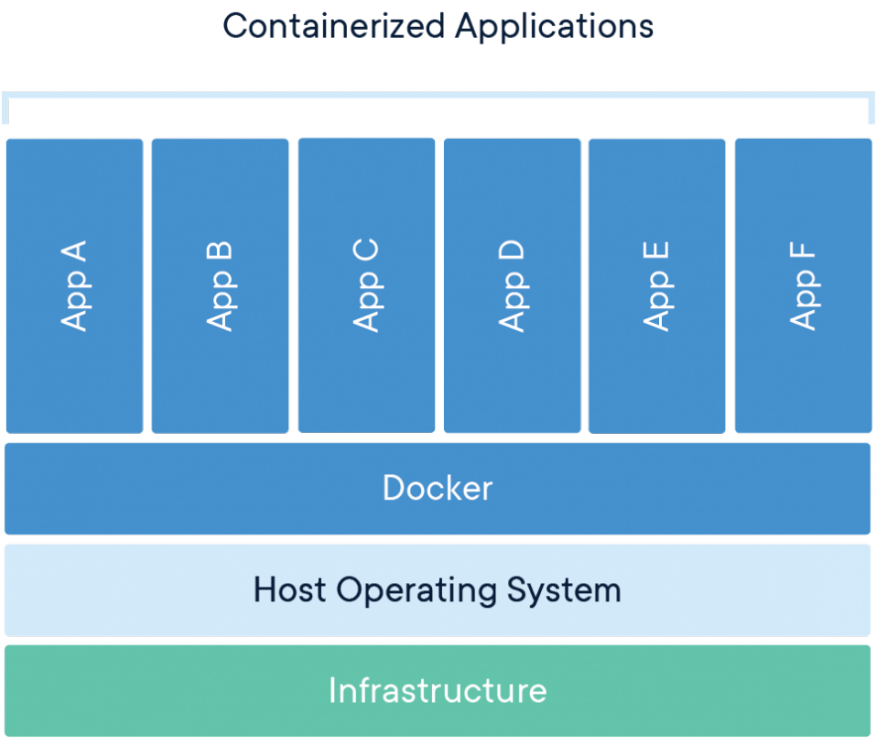
Linux Container (LXC)

- LXC appears to be a light-weight VM running its applications



Docker

- Docker containers created and managed by Docker Engine
- Each container contains a single app



LXC vs Docker

- LXC: lightweight VM, virtual environment, OS/compute centric
- Docker: facilitating app development cycle, app centric
- Taking advantage of kernel features such as namespaces, cgroups for isolation

VM vs LXC vs Docker vs Process

- Your thoughts?

VM vs LXC **Docker vs Process**

Next lecture

- Concurrency and Parallel Processing